PC-Gym: BENCHMARK ENVIRONMENTS FOR PROCESS CONTROL PROBLEMS

Maximilian Bloor[1], **José Torraca**[1,3], **Ilya Orson Sandoval**[1], **Akhil Ahmed**[1], **Martha White**[4]
Mehmet Mercangöz[1], **Calvin Tsay**[2], **Ehecatl Antonio Del Rio Chanona**[1], **Max Mowbray**[1*]
[1]Department of Chemical Engineering, Imperial College London, London, SW7 2AZ, United Kingdom
[2]Department of Computing, Imperial College London, London, SW7 2AZ, United Kingdom
[3]School of Chemistry, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 21941-909, Brazil
[4]Alberta Machine Intelligence Institute (Amii), Department of Computing Science, University of Alberta
{max.bloor22, m.mowbray}@imperial.ac.uk

ABSTRACT

PC-Gym is an open-source tool for developing and evaluating reinforcement learning (RL) algorithms in chemical process control. It features environments that simulate various chemical processes, incorporating nonlinear dynamics, disturbances, and constraints. The tool includes customizable constraint handling, disturbance generation, and reward function design. The tool enables comparison of RL algorithms against nonlinear Model Predictive Control (NMPC) across different scenarios. Case studies demonstrate its effectiveness in evaluating RL approaches for systems like continuously stirred tank reactors, multistage extraction processes, and crystallization reactors. The results reveal performance gaps between RL algorithms and NMPC oracles, highlighting areas for improvement. By providing a standardized platform, PC-Gym aims to accelerate research at the intersection of machine learning and process systems engineering. It connects theoretical RL advances with practical industrial process control applications, offering researchers a tool for exploring data-driven control solutions.

***K*eywords** Reinforcement Learning · Process Control

1 Introduction

In the field of control systems, many problems are addressed using discrete-time approaches, where the system state is sampled (or estimated from output measurements) and control actions are applied at predefined intervals. This paradigm has been successfully employed in various domains, such as robotics, buildings, and automotive systems, enabling the development of effective control strategies. In recent years, model-free reinforcement learning (RL) has become a powerful data-driven framework for such problems. RL allows agents to learn optimal control policies through interaction with their environment, adapting to complex, unknown dynamics and uncertainties [1].

Chemical processes exhibit complex dynamics and constraints that set them apart from traditional RL settings. These processes often involve multiple interacting variables, nonlinear behavior, and time-varying parameters, making them particularly difficult to control using standard RL techniques. Furthermore, chemical processes are often subject to strict safety, environmental, and economic constraints, which must be carefully considered in the control strategy. Violating these constraints can lead to severe consequences, such as equipment damage, product quality degradation, or even human harm.

Due to these complexities, several works have explored the development and application of RL algorithms for chemical process control. Early applications of RL in chemical process control focused on model-free approaches for tracking control and optimization in fed-batch bioreactors [2, 3, 4]ich present unique challenges due to their nonstationary nature and significant run-to-run variabilities [5]. Petsagkourakis et al. [6] introduced a chance-constrained policy optimization algorithm that guarantees the satisfaction of joint chance constraints with high probability. Efforts to improve the scalability of RL algorithms by reducing action-space dimensionality have also shown potential in plantwide

control problems [7]. Works have explored incorporating control structures common in chemical processes such as proportional-integral-derivative (PID) controllers. Lawrence et al. [8] directly parameterized the RL policy as a PID controller instead of using a deep neural network. This allows the RL agent to improve the controller's performance while leveraging existing PID control hardware, demonstrating that industry can utilize actor-critic RL algorithms without additional hardware or sacrificing the interpretability often lost when using deep neural networks. Bloor et al. [9] integrated the neural network and PID controller into a singular RL agent which improved the sample efficiency of the agent compared to a standard deep RL agent due to the embedded control structure. These works all rely on creating discrete control environments, which has motivated the attention of other scientific communities to create open-source accessible environments.

1.1 Related Works

The development of reinforcement learning (RL) algorithms relies heavily on the availability of easily implementable environments that allow for efficient experimentation and iteration. Frameworks such as Gymnasium (formerly known as Gym) by Farama-Foundation [10] and OpenAI's Gym [11] have been instrumental in providing standardized discrete-time environments for RL research, offering a wide range of control problems, games, and simulations. However, these frameworks lack the latest parallelization methods for efficiently learning effective control policies and developing algorithms. To overcome this limitation, several libraries, such as Gymnax [12], Brax [13], Jumanji [14], and PGX [15], have been developed to provide vectorized RL environments in JAX [16], enabling parallel simulations on Graphics Processing Units while maintaining the discrete-time nature of the environments. While these frameworks cover a variety of control problem types, there is still a need for domain-specific environments. Consequently, various natural sciences and engineering communities have started creating custom environments in the same format to simulate their specific problems, such as OR-Gym [17] for operational research and ChemGymRL [18] for chemical discovery. Despite these advancements, there remains a gap in the availability of process control environments with specific functionality tailored to creating discrete-time process control problems.

1.2 Contributions

This work presents PC-Gym, a comprehensive benchmarking suite and development tool for process control algorithms in the chemical industry. The key contributions of this work are as follows:

1. Chemical process control problems: PC-Gym encompasses a variety of unit operations, each considered to represent realistic process dynamics and objectives.
2. Disturbance generation: The suite includes functionality for creating and incorporating disturbances, allowing researchers to test the robustness of their algorithms under various perturbations.
3. Constraint handling: PC-Gym provides mechanisms for incorporating process constraints, encouraging the development of algorithms that respect real-world limitations.
4. Policy visualization and evaluation tools: The suite offers tools for visualizing and evaluating the performance of control policies against a model-based oracle, facilitating the analysis and interpretation of algorithm behavior.

By providing a set of chemical process control scenarios, disturbance, and constraint generation capabilities, and policy visualization and evaluation tools, the suite serves as a valuable resource for researchers and practitioners. The PC-Gym library including code for reproducing the case studies within this work is available at `https://github.com/MaximilianB2/pc-gym`.

The remainder of this paper is organized as follows: Section 2 gives a brief background on reinforcement learning and a formal representation of how PC-Gym represents dynamical systems, Section 3 describes the software implementation of PC-Gym, Section 4 describes the dynamical systems modeled in PC-Gym and benchmarks the performance of common RL algorithms. Finally, Sections 5, 6, and 7 demonstrate PC-Gym's disturbance generation, constraint, and reward function customization functionality respectively.

2 Background

2.1 Dynamical systems

In this work, we assume discrete-time stochastic descriptions of dynamical systems. This is primarily due to the discrete-time nature of modern instrumentation systems, which makes such a framework practical. Specifically, the

state transition is assumed as,

$$X_{t+1} \sim p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)\,, \tag{1}$$

where $\mathbf{x}_t \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ indicates the system states; $\mathbf{u}_t \in \mathcal{U} \subseteq \mathbb{R}^{n_u}$ represents the system's control inputs; the subscript indicates the quantity is measured at timestep t; and $p(\cdot)$ represents a conditional probability distribution. We hypothesize that the conditional probability density could be a representation of a nonlinear state space model,

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t)\,,$$

with stochasticity as the result of general uncertain parameters, $\mathbf{w}_t \in \mathcal{W} \subseteq \mathbb{R}^{n_w}$, indicating disturbance or model parameters, with $\mathcal{W} = \text{supp}(p(\mathbf{w}))$, being the support of its distribution. This work aims to characterize a set of transition distributions $p(\cdot) \in \mathcal{P}$ to represent meaningful process control problems. Each transition function is parameterized in general state-space form, with means to specify different disturbances as the user deems appropriate.

2.2 Learning-based Control

Given a dynamical system as defined in Equation 1, an initial state is sampled from a distribution $X_0 \sim p_0(\mathbf{x}_0)$ where $p_0(\cdot)$ is an assumed initial state distribution. The control problem is defined by the control inputs $\mathbf{u}_t$, which maximize a sequence of rewards, allocated by a function, $r_t : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$. Learning-based control aims to find a sequence of control inputs as a function of the current state based on data collected from a dynamical system over a fixed horizon T. The control problem is approximated with a fixed horizon to create an episodic learning approximation to the infinite horizon task at hand. Therefore, the objective is to find the optimal control policy $\pi^* : \mathcal{X} \to \mathcal{U}$ that maps state to control actions, maximizing the expected cumulative reward.

$$\begin{aligned}
\max_{\pi} \;& \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t r_t(X_t, U_t)\right] \\
s.t. \quad & X_0 \sim p_0(\mathbf{x}_0) \\
& X_{t+1} \sim p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) \\
& U_t \sim \pi(\mathbf{u}_t \mid \mathbf{x}_t) \\
& t \in \{0, \ldots, T\}\,,
\end{aligned}$$

with $\mathbb{E}_{\pi}[\cdot]$ denoting the expectation under the policy π.

2.3 Reinforcement Learning

A popular method to solve the control problem described in the previous section is reinforcement learning (RL), where the major assumption applied is that the system is Markovian. This implies that the problem is modeled as a Markov Decision Process (MDP), and the system's future is independent of the past, given the current state. The reinforcement learning framework (Figure 1) involves an agent and an environment, where the agent observes a state of the environment and inputs an action to the environment, which transitions it to a new state and returns a reward based on a hidden reward function.

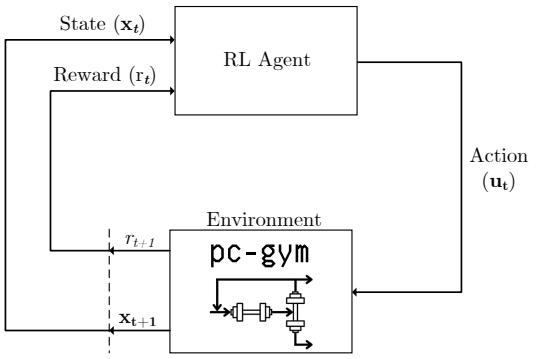

Figure 1: The reinforcement learning framework

Multiple successful types of algorithms have emerged to find the optimal policy in the reinforcement learning framework. In the context of chemical process control, three advanced algorithms have shown particular promise: Proximal Policy

Optimization (PPO), Soft Actor-Critic (SAC), and Deep Deterministic Policy Gradient (DDPG). These algorithms are well-suited to handle the continuous action spaces and complex dynamics often encountered in process control problems. Central to many RL algorithms is the Q-function, also known as the state-action value function. Mathematically, it can be expressed as the expectation of a discounted sum of random variables:

$$Q(\mathbf{x}, \mathbf{u}) = \mathbb{E}_\pi \left[\sum_{t=t'}^{T} \gamma^{t-t'} r_t(X_t, U_t) \middle| X_{t'} = \mathbf{x}, U_{t'} = \mathbf{u} \right], \tag{2}$$

This formulation explicitly shows the Q-function as the expected sum of discounted future rewards, starting from state $\mathbf{x}$ and taking action $\mathbf{u}$, then following the optimal policy for all subsequent steps. Another key concept is the value function $V(\mathbf{x})$, which represents the expected return starting from state $\mathbf{x}$ and following the policy π thereafter. It can be defined as,

$$V^\pi(\mathbf{x}) = \mathbb{E}_\pi \left[\sum_{t=t'}^{T} \gamma^{t-t'} r_t(X_t, U_t) \middle| X_{t'} = \mathbf{x} \right], \tag{3}$$

where $\gamma \in [0, 1]$ is the discount factor. The value function helps in evaluating how "good" it is to be in a particular state, considering the long-term rewards that can be obtained from that state under the current policy. The advantage function, which is used in many RL algorithms, combines the Q-function and the value function:

$$\hat{A}_t = Q(\mathbf{x}_t, \mathbf{u}_t) - V(\mathbf{x}_t)\,. \tag{4}$$

This function estimates the improvement in performance by taking a given control $\mathbf{u}_t$, in state $\mathbf{x}_t$, and following the policy thereafter, rather than simply following the policy. This provides an effective quantity for approximate policy improvement steps.

In practice, these functions are typically parameterized using neural networks due to their ability to handle continuous state and action spaces. The Q-function can be parameterized by parameters θ as $Q_\theta(\mathbf{x}_t, \mathbf{u}_t)$, and the policy as $\pi_\phi(\mathbf{u}_t|\mathbf{x}_t)$ with parameters ϕ. The parameterized policy can be either deterministic, directly outputting actions $\mathbf{u}_t = \pi_\phi(\mathbf{x}_t)$, or stochastic, outputting a probability distribution over actions. For continuous control problems in chemical processes, the policy network often outputs the mean and standard deviation of a Gaussian distribution:

$$\pi_\phi(\mathbf{u}_t|\mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}_t), \boldsymbol{\sigma}_\phi(\mathbf{x}_t)), \tag{5}$$

where $\boldsymbol{\mu}_\phi(\mathbf{x}_t)$ and $\boldsymbol{\sigma}_\phi(\mathbf{x}_t)$ are neural network outputs. The value function is similarly parameterized as $V_\psi(\mathbf{x}_t)$ with parameters ψ. These parameterizations transform the reinforcement learning problem into one of finding optimal parameters θ, ϕ, and ψ through gradient-based optimization. The neural networks are typically trained by minimizing temporal difference errors for the value functions and maximizing expected returns for the policy, subject to various constraints that depend on the specific algorithm employed. In the following, we give a brief description of three learning algorithms that have proved effective policy optimizers within the literature.

2.3.1 Soft Actor-Critic (SAC)

SAC is an off-policy actor-critic method that incorporates entropy regularization into the objective function [19]. The algorithm employs experience replay and uses target networks for both critics with soft updates. It maintains two Q-function networks and one policy network, all implemented as fully connected neural networks. It aims to learn a policy that maximizes both the expected return and the entropy of the policy. The objective function $J(\pi)$ represents the expected return of the policy and is defined as:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r_t(X_t, U_t) \right]. \tag{6}$$

SAC then augments this objective with an entropy term:

$$J_{SAC}(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r_t(X_t, U_t) + \alpha H(\pi) \right], \tag{7}$$

where $H(\pi)$ is the entropy of the policy, and α is a temperature parameter that automatically adjusts through gradient descent to maintain a target entropy level. SAC learns a stochastic policy by outputting the parameters of a Gaussian distribution, which can be beneficial in chemical process environments with multiple optimal solutions or where exploration is crucial.

2.3.2 Deep Deterministic Policy Gradient (DDPG)

DDPG is an off-policy actor-critic algorithm designed specifically for continuous action spaces, making it particularly suitable for process control tasks [20]. For complete implementation details, readers are referred to the original paper. The algorithm implements experience replay and employs target networks for both actor and critic with soft updates. The networks are fully connected, where the actor outputs deterministic actions and the critic network estimates the Q-value of state-action pairs. The actor is updated using the deterministic policy gradient:

$$\nabla_\phi J = \mathbb{E}_{\mathbf{x}_t \sim \rho^\beta} \left[\nabla_a Q(\mathbf{x}, \mathbf{u}; \theta) \mid_{\mathbf{x}=\mathbf{x}_t, \mathbf{u}=\mu(\mathbf{x}_t;\phi)} \nabla_\phi \boldsymbol{\mu}(\mathbf{x}; \phi) \mid_{\mathbf{x}=\mathbf{x}_t} \right] \tag{8}$$

where $\boldsymbol{\mu}(\mathbf{x}; \phi)$ is the deterministic policy and $Q(\mathbf{x}, \mathbf{u}; \theta)$ is the critic's Q-function. The term ρ^β denotes the state distribution under the behavior policy β. This behavior policy adds Ornstein-Uhlenbeck noise to the deterministic actions for exploration. The state distribution ρ^β represents the probability distribution of states encountered when following this behavior policy.

2.3.3 Proximal Policy Optimization (PPO)

PPO is an on-policy policy gradient method that aims to improve the stability and sample efficiency of policy updates [21]. Unlike SAC and DDPG, PPO does not use experience replay or target networks, instead collecting trajectories using the current policy for updates. It uses an actor-network that outputs the mean and standard deviation of a Gaussian policy and a critic network that estimates the value function. PPO introduces a clipped surrogate objective function:

$$L^{CLIP}(\theta_k) = \min \left(\frac{\pi_{\theta_k}}{\pi_{\theta_{k-1}}} \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta_k}}{\pi_{\theta_{k-1}}}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right), \tag{9}$$

where $\frac{\pi_{\theta_k}}{\pi_{\theta_{k-1}}}$ is the probability ratio between the new and old policies, $\hat{A}_t$ is the estimated advantage function calculated using Generalized Advantage Estimation (GAE), and ϵ is a hyperparameter that controls the clipping range. This objective function is a surrogate to Equation 6, which imposes an approximate trust-region and ensures that the policy updates are neither too large nor too small, leading to more stable learning. This is particularly beneficial for highly nonlinear chemical processes.

Throughout this paper, we will use PPO, SAC, and DDPG to demonstrate the flexibility of PC-Gym. These algorithms have been chosen for their effectiveness in dealing with continuous action spaces and their ability to handle the complexities often encountered in chemical process control problems. By using PC-Gym, we aim to provide insights into the relative strengths and weaknesses of these algorithms across different process control scenarios.

2.4 Constrained Reinforcement Learning

In many real-world applications, particularly in chemical process control, the learning-based control problem must consider operational constraints. These constraints may represent safety limits, equipment capabilities, or product quality requirements. To incorporate these constraints, we can reformulate the learning-based control problem introduced in Section 2.2 as follows:

$$\begin{aligned}
\max_{\pi} \quad & \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r_t(X_t, U_t) \right] \\
s.t. \quad & X_0 \sim p_0(\mathbf{x}) \\
& X_{t+1} \sim p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t) \\
& \mathbb{P}\left[g_i(X_t) \leq 0 \right] \geq 1 - \alpha_{i,t}, \quad i \in \{1, \ldots, n_g\} \\
& t \in \{0, \ldots, T\},
\end{aligned}$$

where $g_i(\cdot)$ represents the i-th inequality constraint. The total number of constraints is denoted by n_g. Specifically, the problem imposes an individual chance constraint for each constraint function, given the uncertainty of the state. Explicitly, this allows for constraint violation with a given probability, $\alpha_{i,t}$. It is worth noting that one may also allow a probability for violation of the constraints jointly. This is known as a joint chance constraint. In practice, this is generally enabled by tuning the individual constraints imposed for example through constraint tightening.

This formulation leads us to the field of Constrained Reinforcement Learning (CRL). CRL aims to solve problems where the agent must maximize the expected return while satisfying constraints. Various approaches can be employed

to solve constrained RL problems. These include Lagrangian methods, which transform the constrained optimization problem into an unconstrained one, reward shaping techniques that modify the reward function to incorporate penalties for constraint violations and safe policy optimization algorithms that directly optimize policies while ensuring constraint satisfaction throughout the learning process.

A key metric in evaluating constrained RL algorithms is the probability of constraint violation. Here, we consider the constraints jointly to provide a single metric to evaluate constraint satisfaction. Formally, for a given policy π and constraint function $g_i(\cdot)$, the probability of constraint violation can be defined as:

$$P_{\text{violation}}(\pi) = 1 - \mathbb{P}\left(\bigcap_{t=0}^{T} \{\mathbf{x}_t \in \mathbb{X}_t\}\right), \tag{10}$$

with,

$$\mathbb{X}_t = \{\mathbf{x}_t \in \mathbb{R}^{n_x} \mid g_i(\mathbf{x}_t) \leq 0,\ \forall i \in \{1, \ldots, n_g\}\}.$$

In practice, this probability is often estimated empirically according to an empirical cumulative distribution function defined as the result of multiple episodes or rollouts.

In the context of chemical process control, constrained RL can be particularly valuable for ensuring that learned control policies maintain safe operating conditions, adhere to equipment limitations, and satisfy product quality constraints. The ability to incorporate such constraints directly into the learning process, while quantifying and minimizing the risk of violations, can significantly enhance the practical applicability of RL in industrial settings, hence user-defined constraints and constraint violation information are available for the user to incorporate into their algorithm. Within this work, only reward shaping with constraint violation information is shown since our aim is to demonstrate the flexibility of the package.

2.5 Policy Performance and Reproducibility Measures

In reinforcement learning (RL), assessing the performance of trained policies requires metrics that capture both the performance and the reproducibility of the learned policy. Dispersion metrics play a crucial role in quantifying the variability of policy performance across multiple training runs. While the expected return provides insight into a policy's average performance, metrics like Median Absolute Deviation (MAD) and standard deviation offer valuable information about the dispersion of returns from a policy. These dispersion metrics are particularly important in domains with inherent stochasticity, such as chemical process control, where reproducible behavior is essential.

Given the inherent stochastic nature of chemical systems, the same control policy π can perform differently across runs, which we define as reproducibility [22]. Evaluating both the expected return $\mathbb{E}[J(\pi)|\pi]$ and reproducibility of policies is crucial. PC-Gym addresses this by reporting a performance measure, such as the median expected return $\tilde{J}_\pi$, along with a chosen dispersion metric across multiple training runs for chemical process control environments. This dispersion metric can be either the median absolute deviation $\text{MAD}(J(\pi))$ or the standard deviation $\sigma(J(\pi))$. For instance, the MAD quantifies policy reproducibility as:

$$\text{MAD}(J(\pi)) = \text{median}(|J(\pi) - \tilde{J}(\pi)|)$$

Policies with high $\tilde{J}_\pi$ but poor reproducibility (high dispersion) may be unreliable in settings where consistent performance is critical, while those with slightly lower $\tilde{J}_\pi$ but good reproducibility (low dispersion) may be preferable for reliable operation. Reporting both the performance measure and a dispersion metric enables a more comprehensive and practically relevant evaluation of RL policies in chemical process control.

In addition to performance and reproducibility measures, the optimality gap provides a crucial metric for evaluating reinforcement learning policies in chemical process control. The optimality gap, $\Delta(\pi_\theta)$, quantifies the difference between the performance of a learned policy π_θ and an optimal policy π^*:

$$\Delta(\pi_\theta) = J(\pi^*) - J(\pi_\theta) \tag{11}$$

where $J(\pi_\theta)$ and $J(\pi^*)$ represent the expected returns of the learned and optimal policies, respectively. This metric offers a normalized measure of policy performance across different environments and indicates the potential for improvement. In chemical process control, where true optimal policies may be unknown, upper bounds or benchmark policies such as nonlinear model predictive control with a perfect model (NMPC) often serve as proxies for π^*.

Incorporating the optimality gap alongside performance measures like median expected return $\tilde{J}_\pi$ and dispersion metrics such as MAD or standard deviation provides a comprehensive evaluation framework. This approach balances absolute

performance, consistency, and relative optimality, enabling a more nuanced assessment of RL policies in the stochastic and complex domain of chemical processes. Such an evaluation is essential for developing robust and reliable control strategies that meet the stringent requirements of industrial chemical applications.

2.6 Nonlinear MPC (Oracle)

To establish a performance benchmark for the reinforcement learning algorithms, we implemented a nonlinear Model Predictive Controller (NMPC) as an oracle. This controller leverages complete knowledge of the system dynamics, including perfect state estimation, to solve the optimal control problem at each time step. We define a normalization function for a vector $\mathbf{z}$ as $\bar{\mathbf{z}} = \text{diag}(\mathbf{z}_{max} - \mathbf{z}_{min})^{-1}(\mathbf{z} - \mathbf{z}_{min})$. The NMPC is then formulated as a finite-horizon optimal control problem:

$$\begin{aligned}
\min_{\mathbf{u}_t,\ldots,\mathbf{u}_{t+N-1}} \quad & J(\mathbf{x},\mathbf{u}) = \sum_{k=t}^{t+N} (\bar{\mathbf{x}}_k - \bar{\mathbf{x}}^*)^\top \mathbf{Q} (\bar{\mathbf{x}}_k - \bar{\mathbf{x}}^*) + \sum_{k=t}^{t+N-1} (\bar{\mathbf{u}}_k - \bar{\mathbf{u}}_{k-1})^\top \mathbf{R} (\bar{\mathbf{u}}_k - \bar{\mathbf{u}}_{k-1}) \\
\text{s.t.} \quad & \mathbf{x}_t = \mathbf{x}(t) \\
& \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k), \quad k = t, \ldots, t+N-1 \\
& \mathbf{g}(\mathbf{x}_k) \leq \mathbf{0}, \quad k = t, \ldots, t+N
\end{aligned}$$

In this formulation, $\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k)$ represents a perfect model[2] of the environment. The bar notation (e.g., $\bar{\cdot}_k$) indicates normalized variables, enabling balanced comparison between different variables, regardless of their physical units or magnitudes. The normalized state and setpoint vectors are denoted by $\bar{\mathbf{x}}_k \in \mathbb{R}^{n_x}$, and $\bar{\mathbf{x}}^* \in \mathbb{R}^{n_x}$, respectively; with $\bar{\mathbf{u}}_k \in \mathbb{R}^{n_u}$ denoting the normalized input vector. The positive semi-definite weighting matrices for state and input costs are $\mathbf{Q} \in \mathbb{R}^{n_x \times n_x}$ and $\mathbf{R} \in \mathbb{R}^{n_u \times n_u}$, respectively. The vector-valued constraint function is represented by $\mathbf{g} : \mathbb{R}^{n_x} \to \mathbb{R}^{n_g}$. The subscript k indicates the quantity at timestep k.

The oracle employs the do-mpc [23] framework, which utilizes CasADi [24], to formulate and IPOPT [25] to solve this optimal control problem. The NMPC implementation uses orthogonal collocation to discretize the continuous-time dynamics, allowing for a high-accuracy representation of the system behavior between sampling instants. This approach is particularly beneficial for systems with fast dynamics or when using larger time steps.

The MPC problem is set up with a user-defined horizon length N and control weighting $\mathbf{R}$. The objective function includes terms for both state tracking error and control effort. The state tracking error is computed as the squared difference between each controlled variable's current state and setpoint. The control effort term is calculated using normalized input values to ensure consistent scaling across different input ranges.

This NMPC implementation, with its perfect state estimation and high-accuracy collocation-based discretization, serves as a performance ceiling for the reinforcement learning algorithms. It represents near-optimal control given perfect system knowledge and disturbance prediction within the horizon. By comparing RL algorithm performance against this oracle, we can quantitatively assess their efficacy in approximating optimal control strategies across various process control scenarios.

3 Software Implementation

PC-Gym presents a modular and extensible framework written in Python for simulating fixed control policies for complex dynamical systems. The architecture of PC-Gym (Figure 2) is designed to facilitate the integration of various components essential for reinforcement learning and optimal control tasks while adhering to established standards in the field.

Central to PC-Gym is the `make_env.py` module, which serves as the environment constructor. Crucially, this module implements the Gymnasium [11] interface, a widely adopted standard for reinforcement learning environments. By following the Gymnasium format, PC-Gym ensures compatibility with a broad range of existing RL algorithms and tools. The class accepts user-defined parameters as a dictionary, including model specifications, time steps, setpoints, and constraints.

[2]To be explicit, here a perfect model means that the same realizations of uncertainty observed within the simulator are provided as data to define the nonlinear program. In this sense, the oracle can forecast the future perfectly.

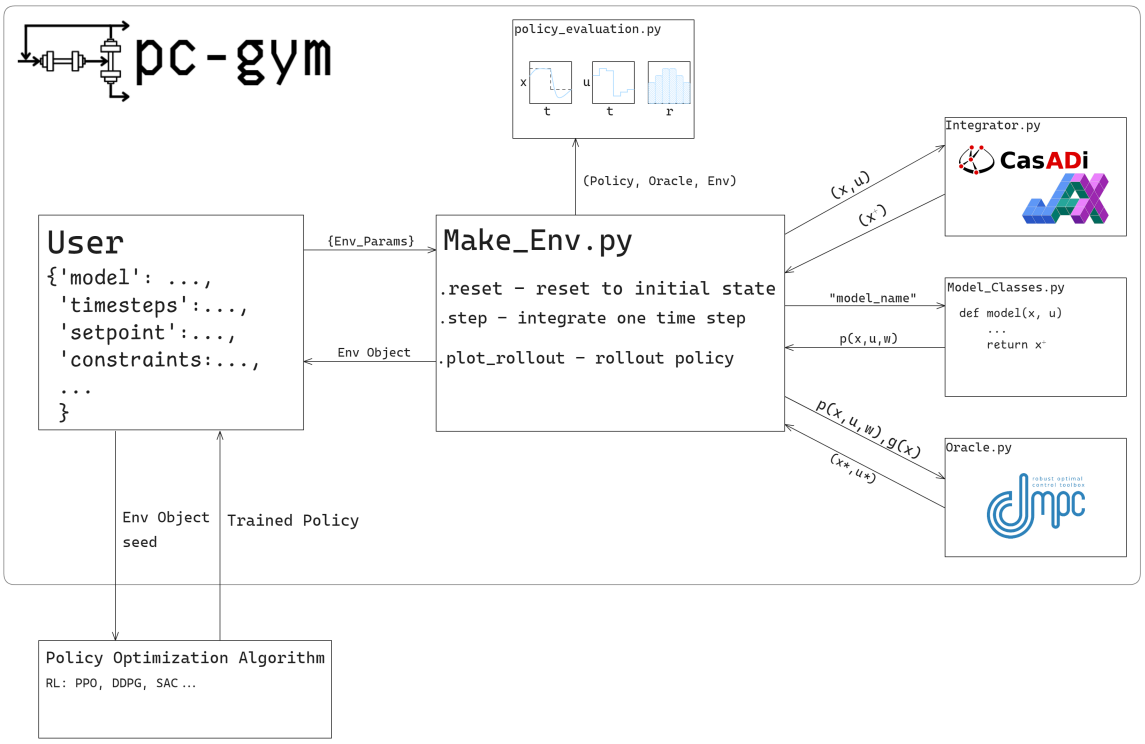

Figure 2: PC-Gym's Architecture

In line with the Gymnasium standard, `make_env.py` provides core methods such as `.reset()` for initializing the environment, and `.step()` for advancing the simulation. This standardization allows researchers and practitioners to seamlessly integrate PC-Gym environments into their existing RL workflows and frameworks.

PC-Gym's modular design incorporates several key components that complement the Gymnasium-compatible environment. The `model_classes.py` defines the dynamics of the controlled system. Numerical integration is handled by `integrator.py`, which utilizes either CASADI [24] or JAX [16] (depending on the user's selection) for efficient computations. The oracle, as described in Section 2.6, is implemented in `oracle.py` using the do-mpc [23] toolkit, while `policy_evaluation.py` manages policy assessment using the performance and dispersion metrics described in Section 2.5. For more detailed information on PC-Gym's features and usage, readers are encouraged to refer to the documentation available at `https://maximilianb2.github.io/pc-gym/`.

4 Control Problems

This section describes four environments included in PC-Gym, indicating the types of dynamical systems included. The uncertainty in each control problem stems from measurement noise which given the perfect state estimator in the Oracle, results in no variation in the oracle's state and control, and therefore cost trajectories. At the end of this section, all environments are benchmarked with three RL algorithms (SAC, PPO, and DDPG) at a setpoint point tracking control problem. All RL algorithms are trained using 50,000 timesteps using the default hyperparameters from Stable Baselines 3 [26] and the same seed. The choice of timestep budget was deemed large for this class of problems, allowing each algorithm to produce the highest performing policy for the same given seed. This policy is then fixed and evaluated on multiple instances of the environment. The following reward function follows the same notation as Oracle's objective function as described in Section 2.6:

$$r_t = -\left((\bar{\mathbf{x}}_t - \bar{\mathbf{x}}^*)^\top \mathbf{Q}(\bar{\mathbf{x}}_t - \bar{\mathbf{x}}^*) + (\bar{\mathbf{u}}_k - \bar{\mathbf{u}}_{k-1})^\top \mathbf{R}(\bar{\mathbf{u}}_k - \bar{\mathbf{u}}_{k-1})\right) \tag{12}$$

where the state and control cost matrices, $\mathbf{R}$ and $\mathbf{Q}$, are set to equal the corresponding matrix in the oracle for each case study.

4.1 Continuously Stirred Tank Reactor

The Continuous Stirred Tank Reactor (CSTR) system [27, 28], a complex and nonlinear dynamical system, is commonly studied in the Process Systems Engineering (PSE) literature. The system involves an irreversible chemical reaction converting species A to B, with cooling provided by water flowing through a coiled tube. Two ordinary differential equations describe the CSTR:

$$\frac{\mathrm{d}C_A}{\mathrm{d}t} = \frac{q}{V}(C_{A_f} - C_A) - kC_A e^{\frac{-E_A}{RT}} \tag{13}$$

$$\frac{\mathrm{d}T}{\mathrm{d}t} = \frac{q}{V}(T_f - T) - \frac{\Delta H_R}{\rho C_p} kC_A e^{\frac{-E_A}{RT}} + \frac{UA}{\rho C_p V}(T_c - T) \tag{14}$$

The system features two state variables: concentration of species A, C_A, and reactor temperature, T. One input control variable, cooling water temperature, T_c. The system's nonlinearities are particularly evident in the exponential term containing the reactor temperature. The CSTR system presents significant challenges for control due to its nonlinear dynamics, strong coupling between state variables, and potential for instability and multiple steady states.

4.2 Computational Implementation

To demonstrate the pc-gym framework, we present the Python implementation used to construct the first case study. This example serves to illustrate how PC-Gym significantly reduces the complexity associated with configuring advanced control environments, thereby enabling researchers to concentrate their efforts on the development and evaluation of control strategies, rather than expending resources on intricate implementation details. The following code segment demonstrates the concise methodology by which a CSTR control problem can be defined and initialized within the PC-Gym framework.

```
import numpy as np
from pcgym import make_env
from custom_reward import sp_track_reward

T, tsim = 60, 25 # 60 timsteps for 25 minutes
x_star = {'Ca': [0.85] * (T // 3) + [0.9] * (T // 3) + [0.87] * (T // 3)} # Setpoints

action_space = {
    'low': np.array([295]),
    'high': np.array([302])
}

observation_space = {
```

```
    'low': np.array([0.7, 300, 0.8]),
    'high': np.array([1, 350, 0.9])
}

# Dictionary which describes the process control problem
env_params = {
    'N': N,
    'tsim': tsim,
    'SP': x_star,
    'o_space': observation_space,
    'a_space': action_space,
    'x0': np.array([0.8, 330, 0.8]),
    'model': 'cstr',
    'noise_percentage': 0.001,
    'custom_reward': sp_track_reward # Define the reward as Equation 11
}

env = make_env(env_params) # environment object describing the process control problem
```

The presented code above exemplifies the encapsulation of fundamental elements of the CSTR control problem within the PC-Gym framework. By precisely defining critical parameters such as the number of timesteps (N), simulation duration (`tsim`), setpoints (`SP`), action and observation spaces, initial conditions (`x0`), and noise levels, researchers can expeditiously configure a realistic and challenging control scenario. We define a custom reward to mimic that shown in Equation 12. The final line of code instantiates the environment using the `make_env` object. This singular operation demonstrates the simplicity with which PC-Gym enables the creation of complex control problems, thereby establishing a robust foundation for subsequent control algorithm development and rigorous testing. This streamlined approach significantly reduces the barriers to entry for researchers in the field of process control.

4.2.1 Results

The performance of three reinforcement learning algorithms was evaluated on the CSTR control problem and compared to the Oracle controller. The oracle used a prediction horizon of 17 steps, an identity Q matrix, and a zero R matrix. An episode was defined as 25 minutes with 60 timesteps. Figure 3 illustrates the control performance of these algorithms.

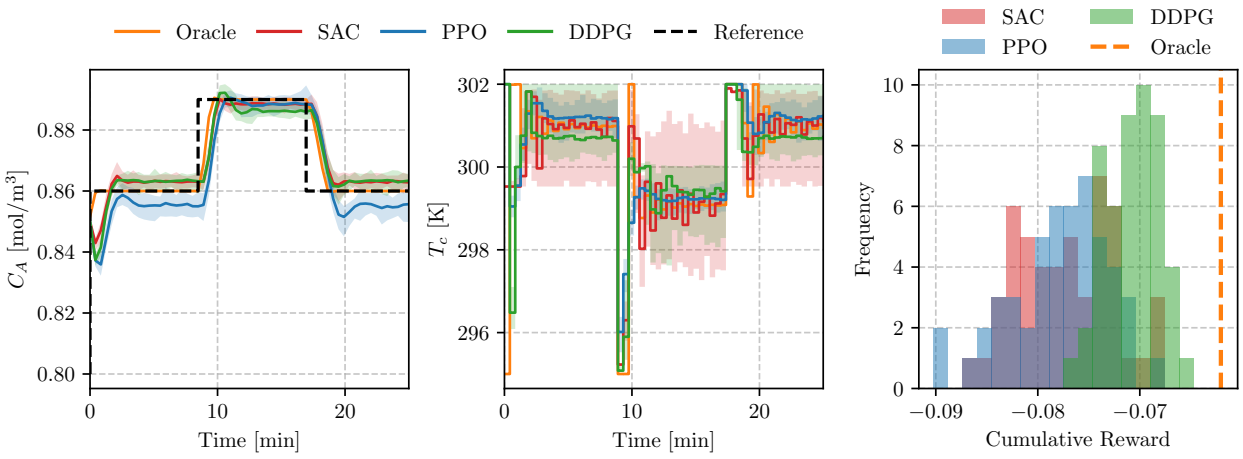

Figure 3: Comparison of reinforcement learning algorithms (SAC, PPO, DDPG) and an oracle for control of the CSTR. Controlled variable response, manipulated variable trajectory, and cumulative reward distribution. Shaded areas show min-max ranges across simulations; solid lines represent median performance.

Figure 3 shows the controlled variable (C_A) response, the manipulated variable (T_C), and the distribution of reward for each algorithm. The oracle is represented as a single dashed line since measurement noise is the only source of

uncertainty and the oracle has a perfect state estimator, therefor there is no variability in its performance. All three RL algorithms demonstrate the ability to track the setpoint changes, with SAC and DDPG showing the closest performance to the oracle. PPO also performs well but exhibits an offset for the first and third setpoints of C_A. SAC's control actions closely resemble the oracle's, with more aggressive transitions and minimal oscillations. DDPG's control actions are smoother than SAC and, with similar setpoint tracking performance, produce a higher reward. This reward histogram provides valuable insights into the performance and consistency of each algorithm. As expected, the oracle achieves the highest rewards with the narrowest distribution, indicating consistent near-optimal performance. DDPG demonstrates the best performance among the RL algorithms, with its reward distribution closest to the oracle. It achieves a high median reward albeit with a large spread similar to the other two reinforcement learning algorithms.

4.3 Multistage Extraction Column

The multistage extraction model [28] describes a counter-current liquid-gas extraction process with five stages. It captures the mass transfer of a solute between the liquid and gas phases in each stage. The model consists of ten ordinary differential equations (ODEs) representing the concentration dynamics of the solute in both phases for each stage. The key equations for stage n, where $n \in 1, 2, 3, 4, 5$, are:

$$\frac{dX_n}{dt} = \frac{1}{V_l}(L(X_{n-1} - X_n) - Q_n) \tag{15}$$

$$\frac{dY_n}{dt} = \frac{1}{V_g}(G(Y_{n+1} - Y_n) + Q_n) \tag{16}$$

where:

$$X_{n,eq} = \frac{Y_n^e}{m} \tag{17}$$

$$Q_n = K_{la}(X_n - X_{n,eq})V_l \tag{18}$$

Here, X_n and Y_n are the solute concentrations in the liquid and gas phases, respectively, L is the liquid flowrate, G is the gas flowrate, Q_n is the mass transfer rate, and $X_{n,eq}$ is the equilibrium concentration. The control variables are L and G, while X_0 (feed concentration of liquid) and Y_6 (feed concentration of gas) are considered disturbances. This model

Table 1: Multistage Extraction Model Parameters

Parameter	Description	Value
V_l	Liquid volume in each stage (m^3)	5.0
V_g	Gas volume in each stage (m^3)	5.0
m	Equilibrium constant (-)	1.0
K_{la}	Mass transfer capacity constant (hr^{-1})	5.0
e	Equilibrium exponent	2.0
X_0	Feed concentration of liquid (kg/m^3)	0.60
Y_6	Feed concentration of gas (kg/m^3)	0.050

represents a complex, multivariable system with nonlinear mass transfer dynamics. Nonlinearity is particularly evident in equilibrium relationships, which are governed by exponential terms. The counter-current nature of the process, coupled with the stage-wise mass transfer, makes this a challenging system for control applications. Typical control objectives might include maintaining desired output concentrations or optimizing extraction efficiency by manipulating the liquid and gas flow rates.

4.3.1 Multistage Extraction Column Results

The performance of three reinforcement learning algorithms was evaluated on the multistage extraction column control problem of the bottom liquid phase X_5 and compared to the Oracle controller. The oracle used a prediction horizon of 20 steps, an identity Q matrix, and a zero R matrix. An episode was defined as 60 hours with 60 timesteps. Figure 4 illustrates the control performance of these algorithms, showing both controlled variables (Y_5, X_5) and manipulated variables (L, G), while Figure 5 displays the distribution of cumulative rewards.

SAC demonstrated the best performance, closely matching the oracle in setpoint, although having different control profiles as SAC utilized the gas flow rate G. In contrast, the oracle maintained it at a constant value. It achieved the highest median reward, indicating near-optimal performance. DDPG performed well as it closely matched the setpoint;

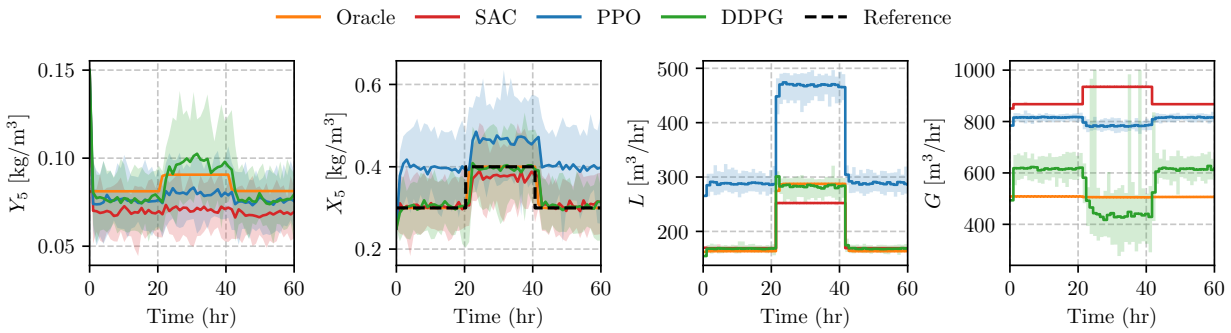

Figure 4: Comparison of reinforcement learning algorithms (SAC, PPO, DDPG) and an oracle for Multistage Extraction Column control. (a-b) Controlled variable response, (c-d) manipulated variable trajectory. Shaded areas show min-max ranges across simulations; solid lines represent median performance.

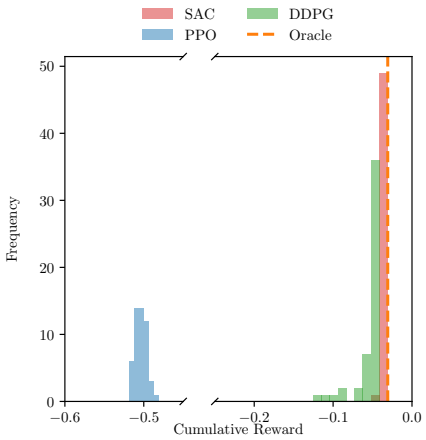

Figure 5: Cumulative reward distributions (100 repetitions) of reinforcement learning algorithms (SAC, PPO, DDPG) and an oracle for Multistage Extraction Column control.

however, aggressive control of the gas flow rate on some of its runs resulted in a wider reward distribution than SAC and the oracle. PPO struggled with setpoint tracking, which had a significant offset throughout the episode, resulting in the poorest performance among the three algorithms. This was reflected in its wide reward distribution and lowest median reward.

4.4 Crystallisation Reactor

The crystallization model [29] describes the population balance for potassium sulfate (K_2SO_4) crystallization using the method of moments. It is a highly nonlinear process represented by a system of ordinary differential equations (ODEs) that capture the evolution of the first four moments of the crystal size distribution (μ_0, μ_1, μ_2, μ_3) and the solute concentration (c). The model incorporates nucleation and growth kinetics, which are influenced by supersaturation and

temperature. The key equations are:

$$\frac{d\mu_0}{dt} = B_0 \tag{19}$$

$$\frac{d\mu_1}{dt} = G_\infty(a\mu_0 + b\mu_1 \times 10^{-4}) \times 10^4 \tag{20}$$

$$\frac{d\mu_2}{dt} = 2G_\infty(a\mu_1 \times 10^{-4} + b\mu_2 \times 10^{-8}) \times 10^8 \tag{21}$$

$$\frac{d\mu_3}{dt} = 3G_\infty(a\mu_2 \times 10^{-8} + b\mu_3 \times 10^{-12}) \times 10^{12} \tag{22}$$

$$\frac{dc}{dt} = -0.5\rho\alpha G_\infty(a\mu_2 \times 10^{-8} + b\mu_3 \times 10^{-12}) \tag{23}$$

where B_0 is the nucleation rate and G_∞ is the crystal growth rate. These rates depend on the supersaturation S and temperature T, which is the control variable:

$$C_{eq} = -686.2686 + 3.579165(T + 273.15) - 0.00292874(T + 273.15)^2 \tag{24}$$

$$S = c \times 10^3 - C_{eq} \tag{25}$$

$$B_0 = k_a \exp\left(\frac{k_b}{T + 273.15}\right)(S^2)^{k_c/2}(\mu_3^2)^{k_d/2} \tag{26}$$

$$G_\infty = k_g \exp\left(\frac{k_1}{T + 273.15}\right)(S^2)^{k_2/2} \tag{27}$$

The model parameters are given in the following table: This model captures the complex crystallization dynamics,

Table 2: Crystallization Model Parameters

Parameter	Description	Value
k_a	Nucleation rate constant	0.92
k_b	Nucleation temperature dependency	-6800
k_c	Nucleation supersaturation exponent	0.92
k_d	Nucleation crystal content exponent	1.3
k_g	Growth rate constant	48
k_1	Growth rate temperature dependency	-4900
k_2	Growth rate supersaturation exponent	1.9
a	Size-dependent growth parameter	0.51
b	Size-dependent growth parameter	7.3
α	Volumetric shape factor	7.5
ρ	Crystal density (g/cm^3)	2.7

including the effects of temperature on nucleation and growth rates, making it a challenging system for control applications. The control objective typically involves manipulating the temperature to achieve desired crystal size distribution characteristics or solute concentration levels.

4.4.1 Crystallization Reactor Results

The performance of three reinforcement learning algorithms was evaluated on the crystalization control problem and compared to the oracle controller. The oracle used a prediction horizon of 2 steps: an identity Q matrix and a zero R matrix. An episode was defined as 30 hours with 30 timesteps. Figure 6 illustrates the control performance of these algorithms, showing both controlled variables (CV, $\bar{L}_n$) and manipulated variable (T_c), this figure also displays the distribution of cumulative rewards.

All three reinforcement learning algorithms perform poorly in tracking $\bar{L}_n$ with a significant overshoot in the first 10 hours compared to the oracle. Out of the three reinforcement learning algorithms, PPO performs the best. This is shown by a higher median reward in the cumulative reward distribution, also shown in Figure 6.

4.5 Four-Tank System

The four-tank system [30] is a multivariable process consisting of four interconnected water tanks. This system is often used as a benchmark for control systems due to its nonlinear dynamics and the coupling between inputs and outputs.

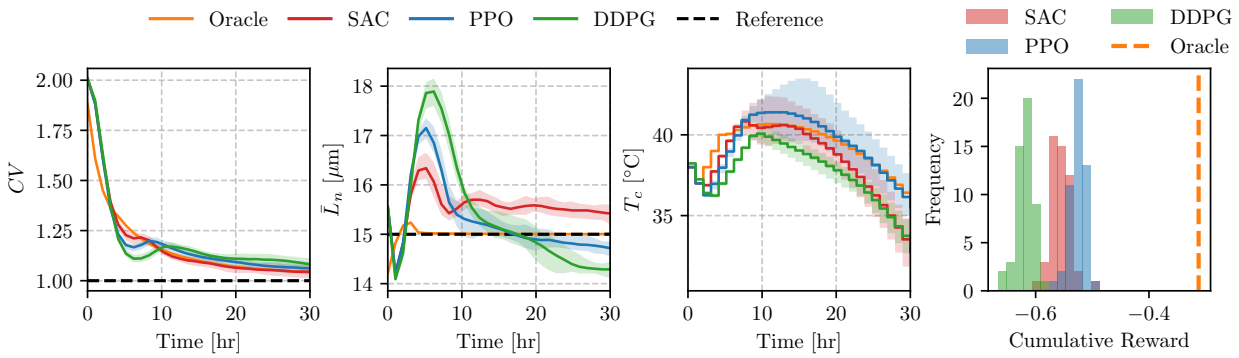

Figure 6: Comparison of reinforcement learning algorithms (SAC, PPO, DDPG) and an oracle for Crystalization Reactor control. (a) Controlled variable response, (b) manipulated variable trajectory, and (c) cumulative reward distribution. Shaded areas show ranges across simulations; solid lines represent median performance.

The model describes the change in water levels in each tank based on the inflows and outflows. The key equations for each tank are:

$$\frac{dh_1}{dt} = -\frac{a_1}{A_1}\sqrt{2g_a h_1} + \frac{a_3}{A_1}\sqrt{2g_a h_3} + \frac{\gamma_1 k_1}{A_1} v_1 \tag{28}$$

$$\frac{dh_2}{dt} = -\frac{a_2}{A_2}\sqrt{2g_a h_2} + \frac{a_4}{A_2}\sqrt{2g_a h_4} + \frac{\gamma_2 k_2}{A_2} v_2 \tag{29}$$

$$\frac{dh_3}{dt} = -\frac{a_3}{A_3}\sqrt{2g_a h_3} + \frac{(1-\gamma_2) k_2}{A_3} v_2 \tag{30}$$

$$\frac{dh_4}{dt} = -\frac{a_4}{A_4}\sqrt{2g_a h_4} + \frac{(1-\gamma_1) k_1}{A_4} v_1 \tag{31}$$

Here, h_i represents the water level in tank i, v_1, and v_2 are the input voltages to the pumps, and the other parameters are described in the table below. This system is characterized by its multivariable nature and the coupling between

Table 3: Four-Tank System Model Parameters

Parameter	Description	Value
g_a	Acceleration due to gravity (m/s^2)	9.8
γ_1	Fraction bypassed by valve to tank 1 (-)	0.20
γ_2	Fraction bypassed by valve to tank 2 (-)	0.20
k_1	1st pump gain (m^3/Volts·s)	8.5×10^{-4}
k_2	2nd pump gain (m^3/Volts·s)	9.5×10^{-4}
a_1	Cross-sectional area of outlet of tank 1 (m^2)	3.5×10^{-3}
a_2	Cross-sectional area of outlet of tank 2 (m^2)	3.0×10^{-3}
a_3	Cross-sectional area of outlet of tank 3 (m^2)	2.0×10^{-3}
a_4	Cross-sectional area of outlet of tank 4 (m^2)	2.5×10^{-3}
A_1	Cross-sectional area of tank 1 (m^2)	1.0
A_2	Cross-sectional area of tank 2 (m^2)	1.0
A_3	Cross-sectional area of tank 3 (m^2)	1.0
A_4	Cross-sectional area of tank 4 (m^2)	1.0

inputs and outputs. The nonlinearity is evident in the square root terms, representing each tank's outflow rates. The system's behavior can be adjusted by changing the valve positions (γ_1 and γ_2), allowing minimum and non-minimum phase configurations. The control objective typically involves regulating the water levels in the lower tanks (h_1 and h_2) by manipulating the input voltages to the pumps (v_1 and v_2). This presents a challenging control problem due to the system's nonlinearity, coupling, and potential for non-minimum phase behavior.

4.5.1 Four-Tank System Results

The performance of three reinforcement learning algorithms was evaluated on the Four Tank system control problem and compared to the Oracle controller. The oracle MPC used a prediction horizon of 17 steps, an identity Q matrix, and a zero R matrix. An episode was defined as 1000 seconds with 60 timesteps. Figure 7 illustrates the control performance of these algorithms, showing both controlled variables ($h1$, $h2$) and manipulated variables (V_1, V_2), while Figure 8 displays the distribution of cumulative rewards.

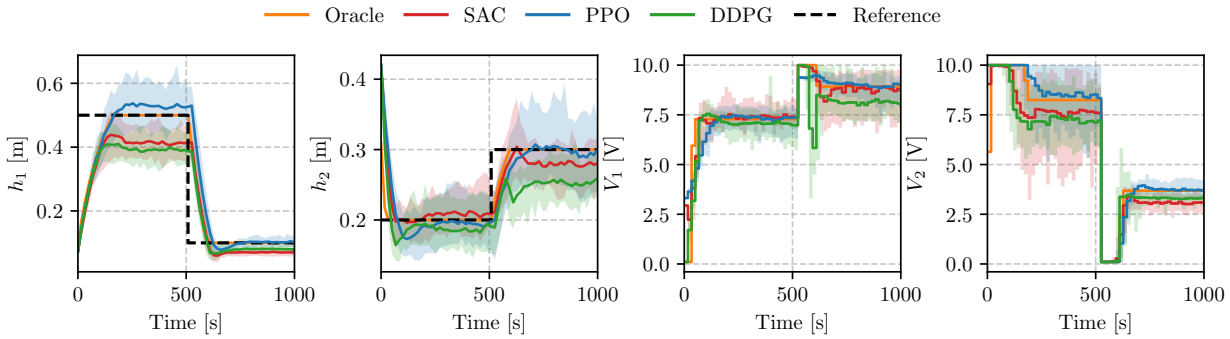

Figure 7: Comparison of reinforcement learning algorithms (SAC, PPO, DDPG) and an oracle for Four Tank System control. Controlled variable responses and manipulated variable trajectories. Shaded areas show min-max ranges across simulations; solid lines represent median performance.

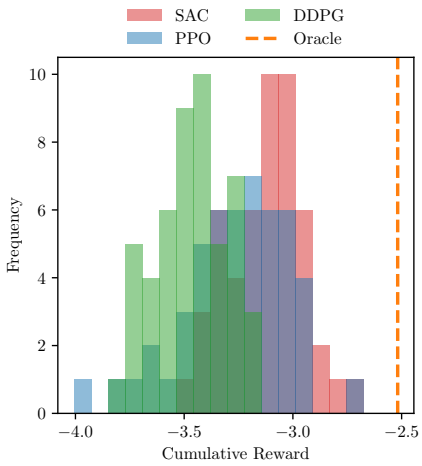

Figure 8: Cumulative reward distributions (20 repetitions) of reinforcement learning algorithms (SAC, PPO, DDPG) and an oracle for Multistage Extraction Column control.

SAC achieved the highest median reward with a narrow distribution, indicating consistent performance albeit with a significant offset for the level of Tank One. PPO showed a lower median reward but with a wide distribution, occasionally reaching SAC's performance levels. DDPG presented the lowest median reward due to significant offset for the first setpoint for h_1 and the second setpoint for h_2.

4.6 Overview of Case Studies

The optimality gap, as previously discussed in Section 2.5, holds significant importance in reinforcement learning for several reasons. Primarily, it offers a scalar measure that quantifies how far a learned policy is from optimality, enabling direct comparisons between different algorithms. For our study in PC-Gym environments, we calculate the empirical optimality gap with normalised cumulative rewards, and with the optimal policy approximated by the oracle $\pi^* \approx \pi_o$:

Table 4: Optimality Gap of SAC, DDPG, and PPO in the PC-Gym Environments

Environment	Optimality Gap		
	DDPG	SAC	PPO
CSTR	**0.008**	0.014	0.0157
Multistage Extraction	0.019	**0.002**	0.474
Crystallization	0.304	0.246	**0.211**
Four Tank System	0.950	**0.579**	0.743

Table 5: Median Absolute Deviation of SAC, DDPG and PPO for the PC-Gym Environments

Policy	Median Absolute Deviation		
	DDPG	SAC	PPO
CSTR	**0.0013**	0.0035	0.0022
Multistage Extraction	0.0021	**0.0001**	0.0053
Crystallization	0.009	0.013	**0.007**
Four Tank	0.124	**0.115**	0.145

$$\hat{\Delta}(\pi_\theta) = \sum_{i=1}^{M} J^{(i)}(\pi_o) - \sum_{i=1}^{M} J^{(i)}(\pi_\theta) \tag{32}$$

where M is the number of episodes, and $r_t^{(i)}(\pi)$ is the reward at time t in episode i under policy π where π_o represents the oracle and π_θ represents an RL policy either optimized with SAC, PPO, or DDPG.

Comparing the optimality gaps (Table 4) across the four PC-Gym environments reveals varying performance among DDPG, SAC, and PPO. DDPG demonstrates superior performance in the CSTR environment with the lowest gap of 0.008. SAC excels in the Multistage Extraction and Four Tank System, achieving gaps of 0.002 and 0.579, respectively. PPO performs best in the Crystallization environment with a gap of 0.211. SAC shows consistent performance across environments, while DDPG and PPO exhibit more variability, suggesting that algorithm selection should be tailored to the specific control task.

The MAD (Table 5) provides insight into the consistency and robustness of each algorithm's performance across multiple runs. Lower MAD values indicate more stable performance. DDPG shows the most consistent performance in the CSTR environment, while SAC demonstrates high stability in the Multistage Extraction process. PPO exhibits the lowest variability in the Crystallization environment. The Four Tank system presents challenges for all algorithms, with SAC showing slightly better consistency. These results highlight the importance of considering both performance and stability when selecting an RL algorithm for process control tasks.

To conclude this case study, our analysis of the optimality gap and median absolute deviation across different PC-Gym environments reveals that no single algorithm consistently outperforms the others in all scenarios. The choice of the most suitable reinforcement learning algorithm depends on the specific characteristics of the process control task at hand. SAC demonstrates a good balance between performance and consistency across various environments, making it a strong general-purpose choice. However, DDPG and PPO show superior performance in specific scenarios, indicating that they may be preferable for certain types of control problems. Furthermore, this case study evaluates fixed policies with a large evaluation budget for training therefore it cannot be said how these algorithms may perform in a sample-constraint and online learning setting. These findings underscore the importance of careful algorithm development required for process control applications, where both performance and stability are crucial factors.

5 Process Disturbances

In chemical process control, disturbances significantly impact system behavior and performance. They can originate from various sources, such as fluctuations in feed composition, environmental changes, or equipment malfunctions. These disturbances can lead to deviations from desired operating conditions, potentially affecting product quality, energy efficiency, and safety. Consequently, the ability to effectively handle and mitigate the impact of disturbances is a key aspect of robust process control.

5.1 Disturbance Generation

To demonstrate the flexibility in disturbance generation of PC-Gym, the CSTR example is used from Section 4.1 with a disturbance in the inlet temperature. To train the agent, an environment was created where an uncertain step change was applied to the inlet temperature after 20 timesteps had passed, then at 40 timesteps the inlet temperature reverted to the original value whilst keeping the reactor temperature at a constant setpoint of 340 K. These inlet temperatures were bounded between 330 K and 370 K. Since SAC and DDPG performed well on the CSTR example in Section 4.2.1 these two algorithms were used to train agents with the default hyperparameters from Stable Baselines 3 [26].

$$T_{in} = \begin{cases} 350 \text{ K} & \text{if } i < 20 \text{ or } i > 40 \\ \mathcal{U}(330 \text{ K}, 370 \text{ K}) & \text{if } 20 \leq i \leq 40 \end{cases} \tag{33}$$

where $\mathcal{U}(L, U)$ represents a 1D uniform distribution with a lower bound of L and upper bound of U. Both SAC and DDPG were used to train an agent on this distribution of disturbances for 50,000 timesteps each. One disturbance from the training distribution and one outside of each algorithm were chosen to test each algorithm.

5.2 Computational Implementation

To generate the disturbances for this experiment in PC-Gym the user will have to define two dictionaries. The first is the disturbance bounds which define the disturbance space that completes the observation space. his dictionary specifies the lower and upper bounds for each disturbance variable. The second is the disturbance dictionary, which employs disturbance variable names as keys and their corresponding time-indexed values as entries. This structure allows users to specify the evolution of disturbances over time. These disturbances are treated as additional control inputs to the model, which either take the model's default value or the user's disturbance value. This retains flexibility in the definition of the disturbance. For this experiment, the computational implementation of the environment creation (parameters not unique to disturbance generation were omitted for readability) is as follows:

```
import numpy as np
from pcgym import make_env

# Define disturbance bounds
disturbance_bounds = {
    'low': np.array([330]),
    'high': np.array([370])
}


# Environment parameters
env_params_template = {
                        # ...other parameters ...
                        'disturbance_bounds': disturbance_bounds
                        # Disturbance generated from uniform distribution
                        'disturbance': {'Ti': [350] * (T // 3)
                                            + [np.random.uniform(330, 370)] * (T // 3)
                                            + [350] * (T // 3)}
                        # ... other parameters ...
                        }

env = make_env(env_params) # environment object describing the process control problem
```

5.3 Results

The agents are first tested on one sample from the training distribution of disturbance, the resulting controlled variable and manipulated variable trajectories for both RL agents and the oracle are shown in Figure 9. The disturbance shown in this figure is a step change of 350 K to 363 K of the inlet temperature. Both the RL agents exhibit significant offset throughout the simulation compared to the oracle, which can track the setpoint with negligible error. However, when the

disturbance occurs, the agent trained with SAC performs aggressive control actions to attempt to reject the disturbance, whereas the agent trained with DDPG performs more conservative control, which results in less effective disturbance rejection. This can be quantified with the optimality gap, where over 10 samples of the training distribution, the agent trained with SAC has a mean normalized optimality gap of 76.58 compared to 138.97 for the agent trained with DDPG.

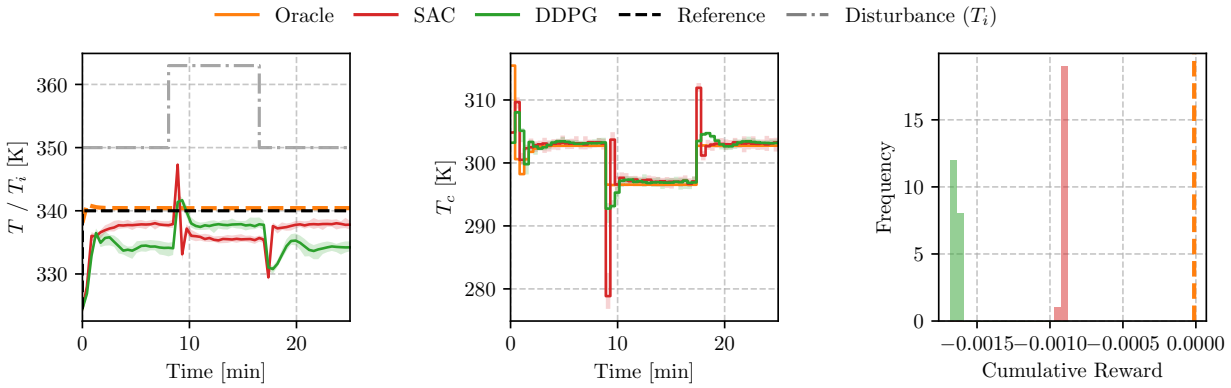

Figure 9: Controlled variable and disturbance trajectories, the manipulated variable trajectories, and cumulative reward distribution for a step change of 350 K to 363 K to the inlet temperature.

Both agents were then tested on a disturbance from outside of the training distribution, and this was selected as a step change to 375 K. The resulting controlled and manipulated variable trajectories, along with the distribution of rewards for both agents and the oracle, are shown in Figure 10. The policy learned using SAC, which was effective within the training distribution, now results in an overly aggressive control action, which leads to a large spike in reactor temperature. On the other hand, the policy trained with DDPG rejects the disturbance more effectively, given its conservative control actions. The ranking of agents in terms of optimality gap swaps position with the agent trained with DDPG performing better with a normalized optimality gap on this disturbance of 408.3 compared to 525.08 for the agent trained with SAC.

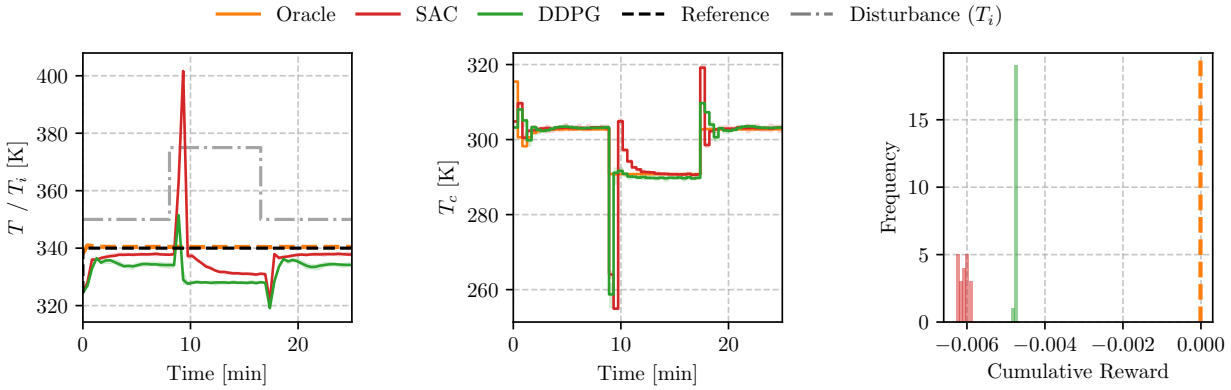

Figure 10: Controlled variable and disturbance trajectories, the manipulated variables trajectories, and cumulative reward distribution for a step change of 350 K to 375 K to the inlet temperature.

The demonstration of PC-Gym's disturbance generation capabilities in the CSTR example highlights its flexibility and utility in training robust control agents. By allowing users to define time-indexed disturbance values, PC-Gym enables the creation of realistic and challenging scenarios for reinforcement learning in process control. The ability to easily implement and evaluate various disturbance patterns in PC-Gym will accelerate the development of adaptive and robust process control using reinforcement learning techniques.

6 Constraint Handling

In chemical process control, operating within safe and efficient boundaries is crucial. Our library implements a flexible constraint-handling mechanism to model these real-world limitations effectively. This section demonstrates constraint's impact on RL in chemical process environments. Here, the focus is to demonstrate the flexibility of PC-Gym in imposing state constraints. Many constrained-RL algorithms have been developed, which could be deployed [31, 32, 33]. In the following, we demonstrate a simple reward shaping strategy for the purpose of demonstration.

6.1 Reward Shaping for Constraint Satisfaction

To encourage constraint satisfaction without hard termination, we implement a custom reward function that incorporates a significant penalty for violations:

$$r_{con}(\mathbf{x}_t, \mathbf{u}_t) = r(\mathbf{x}_t, \mathbf{u}_t) - \lambda \sum_{i=0}^{n_g} \max(0, g_i(\mathbf{x}_t, \mathbf{u}_t)) \tag{34}$$

where r is the base reward function as described in Equation 12, $\lambda > 0$ is a penalty coefficient, and the sum term represents the cumulative constraint violation. This approach effectively shapes the reward landscape, guiding the RL agent towards constraint-satisfying policies while maintaining exploration in the full state space. This is not the only way to handle constraints within the reinforcement learning framework, for further reading we refer to works within the safe RL community. This approach was used to demonstrate the constraint-creation capabilities of PC-Gym.

6.2 Computational Implementation

The creation of the environment used in the constraint showcase is shown below. First, the constraints and their type are defined as dictionaries where the key is the constrained state, and the value is either a list of constraints and types. Then these are passed to the environment parameter dictionary and subsequently to the `make_env` class. The constraint violation information for the current timestep can be accessed from the tuple returned by the `.step` method of the environment object.

```
import numpy as np
from pcgym import make_env
from custom_reward import con_reward

# Constraints and their type are defined
cons = {'T':[327,321]}
cons_type = {'T':['<=','>=']}

env_params_con = {
    ...
    'constraints': cons, # Constraints are added to the environment
    'cons_type': cons_type, # Constraint types are added to the environment
    ...
    }

env_con = make_env(env_params_con) # Environment object describing the process control problem
```

6.3 Experimental Results

We conducted experiments using the CSTR environment introduced in Section 4.1 to evaluate our constraint mechanism. Two DDPG agents were trained: one with reward shaped according to Equation 34 with $\lambda = 1$ to equally weight constraint satisfaction and setpoint tracking, and one with the base reward function as shown in Equation 12. The environment simulated a typical CSTR operation with setpoint changes in reactant concentration and included temperature constraints of the form:

$$321\,\mathrm{K} \leq T \leq 327\,\mathrm{K} \tag{35}$$

Figure 11 shows the rollout results for both agents, focusing on the temperature state variable and control actions. The

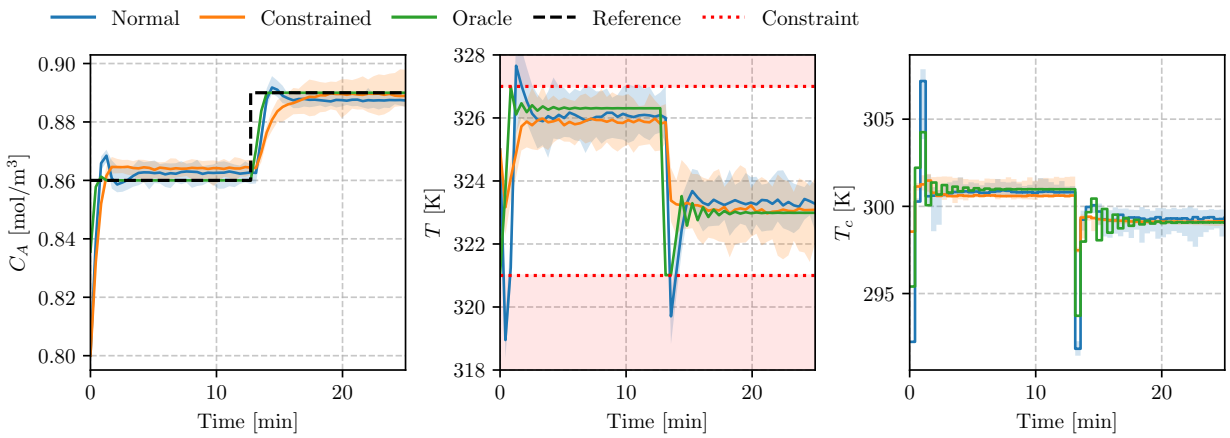

Figure 11: Rollout comparison of constrained and unconstrained DDPG agents over 50 repetitions

constrained agent successfully learned to maintain the temperature within the specified bounds, as evident from the temperature trajectory in Figure 11. In contrast, the base agent violated these limits in pursuit of optimal setpoint tracking for the concentration (Figure 11), especially in the overshoot of both setpoints. To quantify constraint satisfaction, we leverage the empirical cumulative probability of constraint violation, V:

$$P_{violation}(\pi) = \frac{1}{N}\sum_{i=1}^{N} I(g(\tau_i) > 0) \tag{36}$$

where N is the total number of episodes, I is the indicator function that evaluates the entire trajectory of an episode, and $\tau_i = (\mathbf{x}_t, \mathbf{u}_t)_{t=0}^{T}$ represents the complete state-action trajectory of the i-th episode from time step 0 to T. The violation metric results for the experiment are shown in Table 6.

Table 6: Empirical constraint violation probability for the base reward, shaped reward, and the Oracle

Policy	Empirical Violation Probability (V)
Base reward	0.0727
Shaped reward	0.0003
Oracle	0.0000

From the violation metric, the shaping of the reward significantly reduces the probability compared to the base case. However, there is still a substantial optimality gap to the oracle of 0.016. This shows that the RL agent could operate less conservatively and improve setpoint tracking whilst satisfying the constraints. These results demonstrate that PC-Gym's constraint handling mechanism provides a robust foundation for studying and implementing constrained RL in chemical process control. It offers the flexibility necessary for realistic scenarios while enabling systematic comparison of different safe and efficient learning approaches.

7 Reward Functions

In process control applications, the optimal control strategy often depends on the specific goals and limitations of the system. For example, in a CSTR control task, the reward function might prioritize maintaining the reactor concentration within a certain range, minimizing energy consumption, or reducing the variability of the controlled variable. By allowing users to define their reward functions, PC-Gym provides the flexibility to model the requirements of the process control problem. To showcase this flexibility, we will demonstrate three ways to design reward functions for a setpoint tracking task: absolute error, square error, and a sparse reward formulation. The first formulation, negative absolute error (37), will reward the agent for being close to setpoint ($\mathbf{x}^*$) and is defined by:

$$r_t = -||\bar{\mathbf{x}}^* - \bar{\mathbf{x}}||_1 \tag{37}$$

The negative squared error (38) will also reward the agent for being close to the setpoint. However, it will penalize the agent more heavily than the absolute error as it moves away from the set point.

$$r_t = -||\bar{\mathbf{x}}^* - \bar{\mathbf{x}}||_2 \tag{38}$$

Furthermore, a sparse reward (39) can be defined where a reward is only given when the squared setpoint error is smaller than a threshold value ϵ.

$$r(t) = \begin{cases} 1, & \text{if } -||\bar{\mathbf{x}}^* - \bar{\mathbf{x}}||_2 < \epsilon \\ 0, & \text{otherwise} \end{cases} \tag{39}$$

7.1 Computational Implementation

The following Python code was used to implement the training of three different reward functions within PC-Gym. First, a list of reward functions is defined which is then looped over to create an environment and subsequent policy with each reward function. The code unique to the creation of this experiment is shown below:

```
from pcgym import make_env
from custom_reward import r_sparse, r_squared, r_abs

# Define a list of reward functions
r_func = [r_squared,r_sparse, r_abs]

# loop through the custom reward functions
for r_func_i in r_func:
  env_params = {
                # ... other parameters ...
                'custom_reward': r_func_i
                # ... other parameters ...
                }

    env = make_env(env_params)
```

7.2 Results

These three reward functions are implemented with PC-Gym's custom reward functionality on the CSTR environment with ϵ set to 0.003 for the sparse reward function. Three DDPG reinforcement learning agents are all trained for 15k timesteps with the default hyperparameters from Stable Baselines 3 [26]. The trajectories sampled from the current policy every 500 timesteps of training are shown in Figure 12.

Figure 12 shows that different reward functions designed for the same task, setpoint tracking, can produce significantly different policies. The sparse reward signal can make it more difficult for the agent to learn, especially in the early stages of training when the agent rarely receives a non-zero reward, and it cannot further improve the setpoint error below the threshold value provided. The square error leads to slower learning when the agent is close to the setpoint. In contrast, the absolute error provides a consistent reward signal proportional to the setpoint error, allowing it to track the setpoint well.

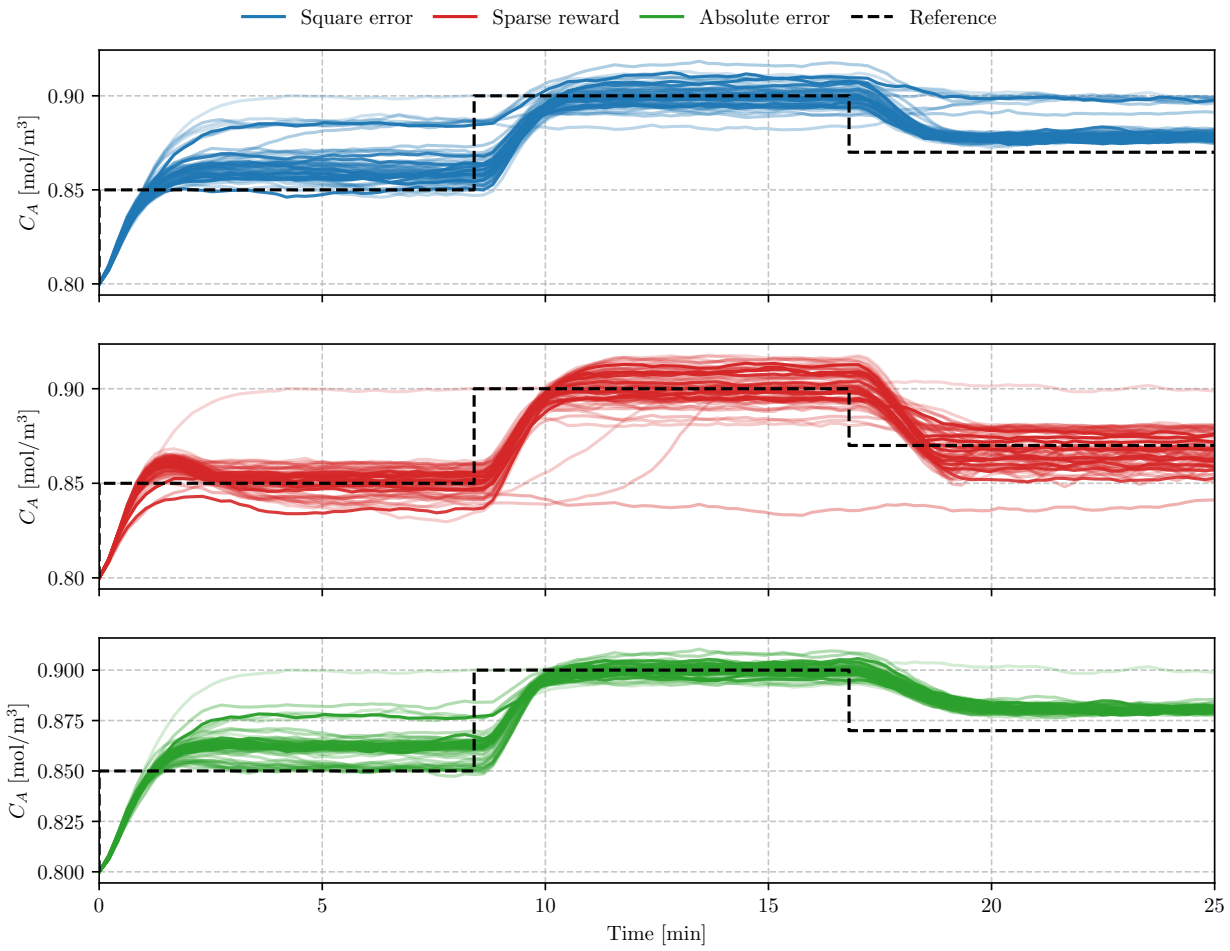

Figure 12: Comparison of learning progress for different reward functions in DDPG. The plot shows the concentration of reactant A (C_A) over time for three reward strategies: square error (top), sparse reward (middle), and absolute error (bottom). The dashed black line represents the reference trajectory. Lighter colors show later iterations.

8 Conclusion and future works

PC-Gym provides a comprehensive framework for applying reinforcement learning to chemical process control problems. Offering discrete-time environments with nonlinear dynamics, flexible constraint handling, disturbance generation, and customizable reward functions enables the evaluation of RL algorithms against traditional control methods. The framework's modular design makes it straightforward for users to implement custom environments, allowing researchers to easily extend PC-Gym's capabilities to their specific process control scenarios while maintaining compatibility with the core framework features. Our benchmarking results across various process scenarios demonstrate PC-Gym's effectiveness in assessing RL approaches for complex control tasks. This framework bridges the gap between theoretical RL advancements and practical applications in industrial process control, paving the way for more improved control strategies. PC-Gym aims to accelerate research within the application of RL to chemical process control, offering a standardized platform for developing and testing RL-based control strategies in realistic chemical process environments.

Future work on PC-Gym could expand its chemical process model library to include more diverse and complex chemical systems. This focus would call on the chemical process control community to contribute their models to improve the applicability of PC-Gym. Additionally, only fixed policies trained with large evaluation budgets were evaluated in this work hence only demonstrating how an algorithm performs under an ideal case of evaluations. Further work could be done to implement an online learning feature in the PC-Gym to allow the testing of algorithms where sample efficiency is paramount. Lastly, creating interfaces for real-time data streaming and hardware-in-the-loop testing would facilitate the transition from simulation to real-world implementation of RL-based control strategies.

9 Acknowledgements

Maximilian Bloor would like to acknowledge funding provided by the Engineering & Physical Sciences Research Council, United Kingdom, through grant code EP/W524323/1. Calvin Tsay acknowledges support from a BASF/Royal Academy of Engineering Senior Research Fellowship. José Torraca would like to acknowledge funding provided by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018. ISBN: 0262039249.

[2] Niket S Kaisare, Jong Min Lee, and Jay H Lee. "Simulation based strategy for nonlinear optimal control: Application to a microbial cell reactor". In: *International Journal of Robust and Nonlinear Control: IFAC-Affiliated Journal* 13.3-4 (2003), pp. 347–363.

[3] Catalina Valencia Peroni, Niket S Kaisare, and Jay H Lee. "Optimal control of a fed-batch bioreactor using simulation-based approximate dynamic programming". In: *IEEE Transactions on Control Systems Technology* 13.5 (2005), pp. 786–790.

[4] JA Wilson and EC Martinez. "Neuro-fuzzy modeling and control of a batch process involving simultaneous reaction and distillation". In: *Computers & chemical engineering* 21 (1997), S1233–S1238.

[5] Haeun Yoo et al. "Reinforcement learning for batch process control: Review and perspectives". In: *Annual Reviews in Control* 52 (2021), pp. 108–119. ISSN: 1367-5788. DOI: https://doi.org/10.1016/j.arcontrol.2021.10.006. URL: https://www.sciencedirect.com/science/article/pii/S136757882100081X.

[6] Panagiotis Petsagkourakis et al. "Chance constrained policy optimization for process control and optimization". In: *Journal of Process Control* 111 (2022), pp. 35–45. ISSN: 0959-1524. DOI: https://doi.org/10.1016/j.jprocont.2022.01.003. URL: https://www.sciencedirect.com/science/article/pii/S0959152422000038.

[7] Lingwei Zhu et al. "Scalable reinforcement learning for plant-wide control of vinyl acetate monomer process". In: *Control Engineering Practice* 97 (2020), p. 104331.

[8] Nathan P Lawrence et al. "Deep reinforcement learning with shallow controllers: An experimental application to PID tuning". In: *Control Engineering Practice* 121 (2022), p. 105046.

[9] Maximilian Bloor et al. *Control-Informed Reinforcement Learning for Chemical Processes*. 2024. arXiv: 2408.13566 [eess.SY]. URL: https://arxiv.org/abs/2408.13566.

[10] Mark Towers et al. *Gymnasium*. Mar. 2023. DOI: 10.5281/zenodo.8127026. URL: https://zenodo.org/record/8127025 (visited on 07/08/2023).

[11] Greg Brockman et al. *OpenAI Gym*. 2016. eprint: arXiv:1606.01540.

[12] Robert Tjarko Lange. *gymnax: A JAX-based Reinforcement Learning Environment Library*. Version 0.0.4. 2022. URL: http://github.com/RobertTLange/gymnax.

[13] C. Daniel Freeman et al. *Brax - A Differentiable Physics Engine for Large Scale Rigid Body Simulation*. Version 0.10.5. 2021. URL: http://github.com/google/brax.

[14] Clément Bonnet et al. *Jumanji: a Diverse Suite of Scalable Reinforcement Learning Environments in JAX*. 2024. arXiv: 2306.09884 [cs.LG]. URL: https://arxiv.org/abs/2306.09884.

[15] Sotetsu Koyamada et al. "Pgx: Hardware-Accelerated Parallel Game Simulators for Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 45716–45743.

[16] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[17] Christian D. Hubbs et al. *OR-Gym: A Reinforcement Learning Library for Operations Research Problems*. 2020. eprint: arXiv:2008.06319.

[18] Chris Beeler et al. "ChemGymRL: A customizable interactive framework for reinforcement learning for digital chemistry". In: *Digital Discovery* 3 (4 2024), pp. 742–758. DOI: 10.1039/D3DD00183K. URL: http://dx.doi.org/10.1039/D3DD00183K.

[19] Tuomas Haarnoja et al. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. 2018. arXiv: 1801.01290 [cs.LG].

[20] Timothy P. Lillicrap et al. *Continuous control with deep reinforcement learning*. 2019. arXiv: 1509.02971 [cs.LG]. URL: https://arxiv.org/abs/1509.02971.

[21] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

[22] Manon Flageat, Bryan Lim, and Antoine Cully. *Beyond Expected Return: Accounting for Policy Reproducibility when Evaluating Reinforcement Learning Algorithms*. 2024. arXiv: 2312.07178 [cs.LG]. URL: https://arxiv.org/abs/2312.07178.

[23] Felix Fiedler et al. "do-mpc: Towards FAIR nonlinear and robust model predictive control". In: *Control Engineering Practice* 140 (2023), p. 105676. ISSN: 0967-0661. DOI: https://doi.org/10.1016/j.conengprac.2023.105676. URL: https://www.sciencedirect.com/science/article/pii/S0967066123002459.

[24] Joel A E Andersson et al. "CasADi – A software framework for nonlinear optimization and optimal control". In: *Mathematical Programming Computation* 11.1 (2019), pp. 1–36. DOI: 10.1007/s12532-018-0139-4.

[25] Andreas Wächter and Lorenz T. Biegler. "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". In: *Mathematical Programming* 106.1 (Mar. 2006), pp. 25–57. ISSN: 1436-4646. DOI: 10.1007/s10107-004-0559-y. URL: https://doi.org/10.1007/s10107-004-0559-y.

[26] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: http://jmlr.org/papers/v22/20-1364.html.

[27] Dale E. Seborg et al. *Process Dynamics and Control*. 4th. John Wiley and Sons, 2011.

[28] John Ingham et al. *Chemical Engineering Dynamics*. 3rd. Wiley, 2007.

[29] Marcellus Guedes Fernandes de Moraes et al. "Modeling and Predictive Control of Cooling Crystallization of Potassium Sulfate by Dynamic Image Analysis: Exploring Phenomenological and Machine Learning Approaches". In: *Industrial & Engineering Chemistry Research* 62.24 (June 2023). Publisher: American Chemical Society, pp. 9515–9532. ISSN: 0888-5885. DOI: 10.1021/acs.iecr.3c00739. URL: https://doi.org/10.1021/acs.iecr.3c00739.

[30] Karl Henrik Johansson. "The quadruple-tank process: A multivariable laboratory process with an adjustable zero". In: *IEEE Transactions on Control Systems Technology* 8 (3 2000). ISSN: 10636536. DOI: 10.1109/87.845876.

[31] Yiming Zhang, Quan Vuong, and Keith Ross. "First order constrained optimization in policy space". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 15338–15349.

[32] Panagiotis Petsagkourakis et al. "Chance constrained policy optimization for process control and optimization". In: *Journal of Process Control* 111 (2022), pp. 35–45.

[33] Max Mowbray et al. "Safe chance constrained reinforcement learning for batch process control". In: *Computers & chemical engineering* 157 (2022), p. 107630.